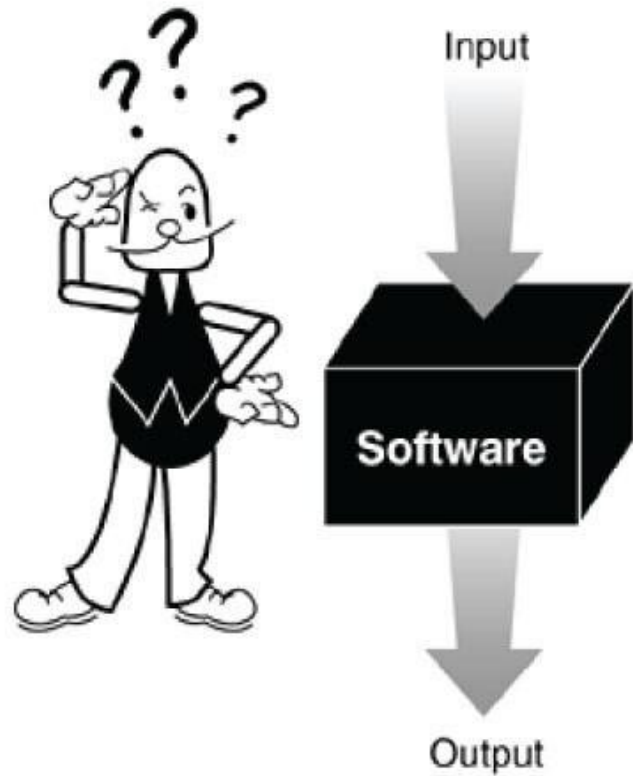


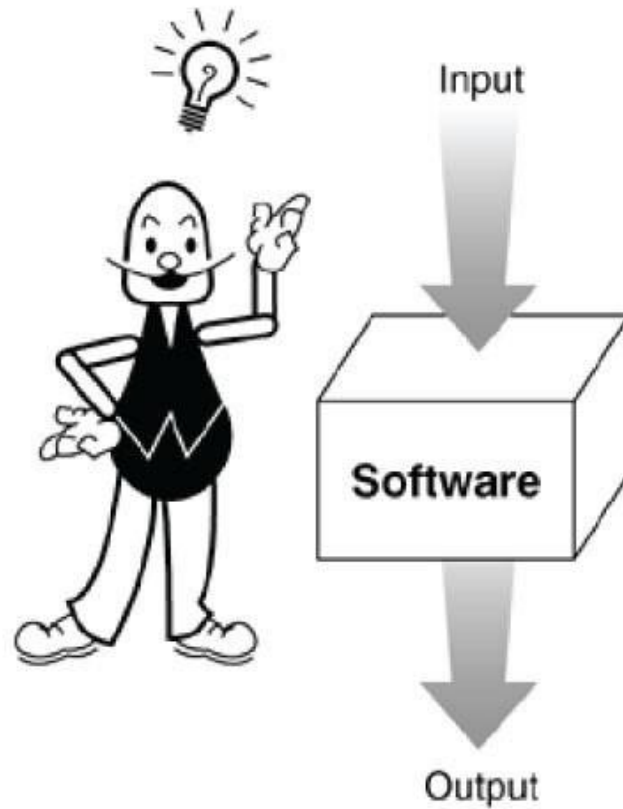


Software Engineering

LECTURE 14: BLACK BOX TESTING



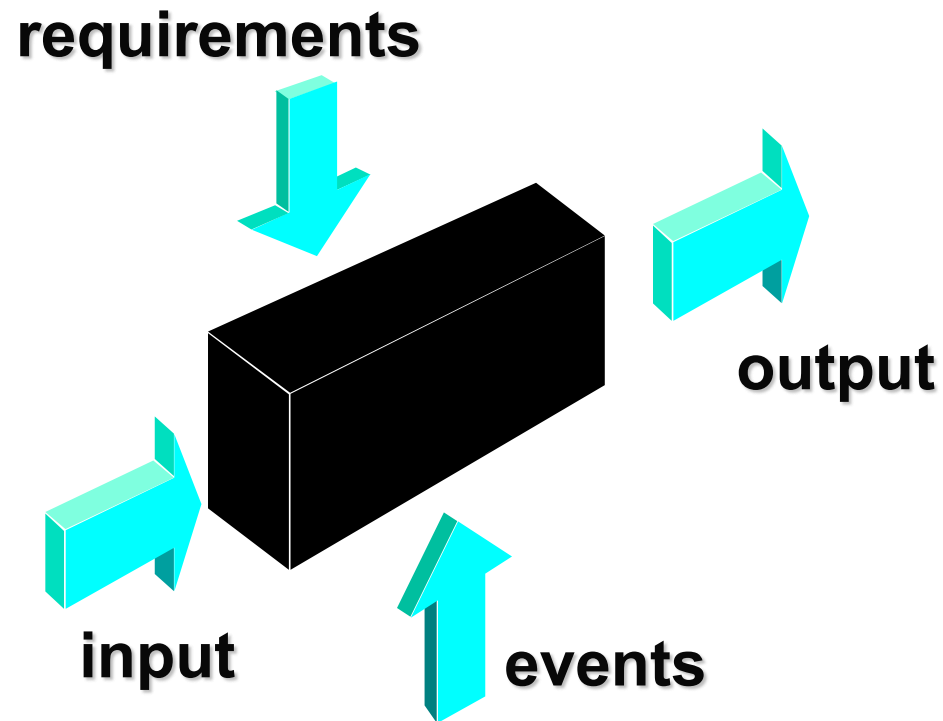
Black-Box Testing



White-Box Testing

White Box Testing vs Black Box Testing

Black-Box Testing



EC-1-Equilateral triangle

($\langle x, y, z \rangle$; $x=y=z$)

EC-2-Isosceles triangle

($\langle x, y, z \rangle$; $x=y, x \neq z$)

EC-3

($\langle x, y, z \rangle$; $x=z, x \neq y$)

EC-4

($\langle x, y, z \rangle$; $y=z, x \neq y$)

EC-5-Scalene triangle

($\langle x, y, z \rangle$; $x \neq y, x \neq z, y \neq z$)

EC-6-Not a triangle

($\langle x, y, z \rangle$; $x \geq y+z$)

EC-7

($\langle x, y, z \rangle$; $y \geq x+z$)

EC-8

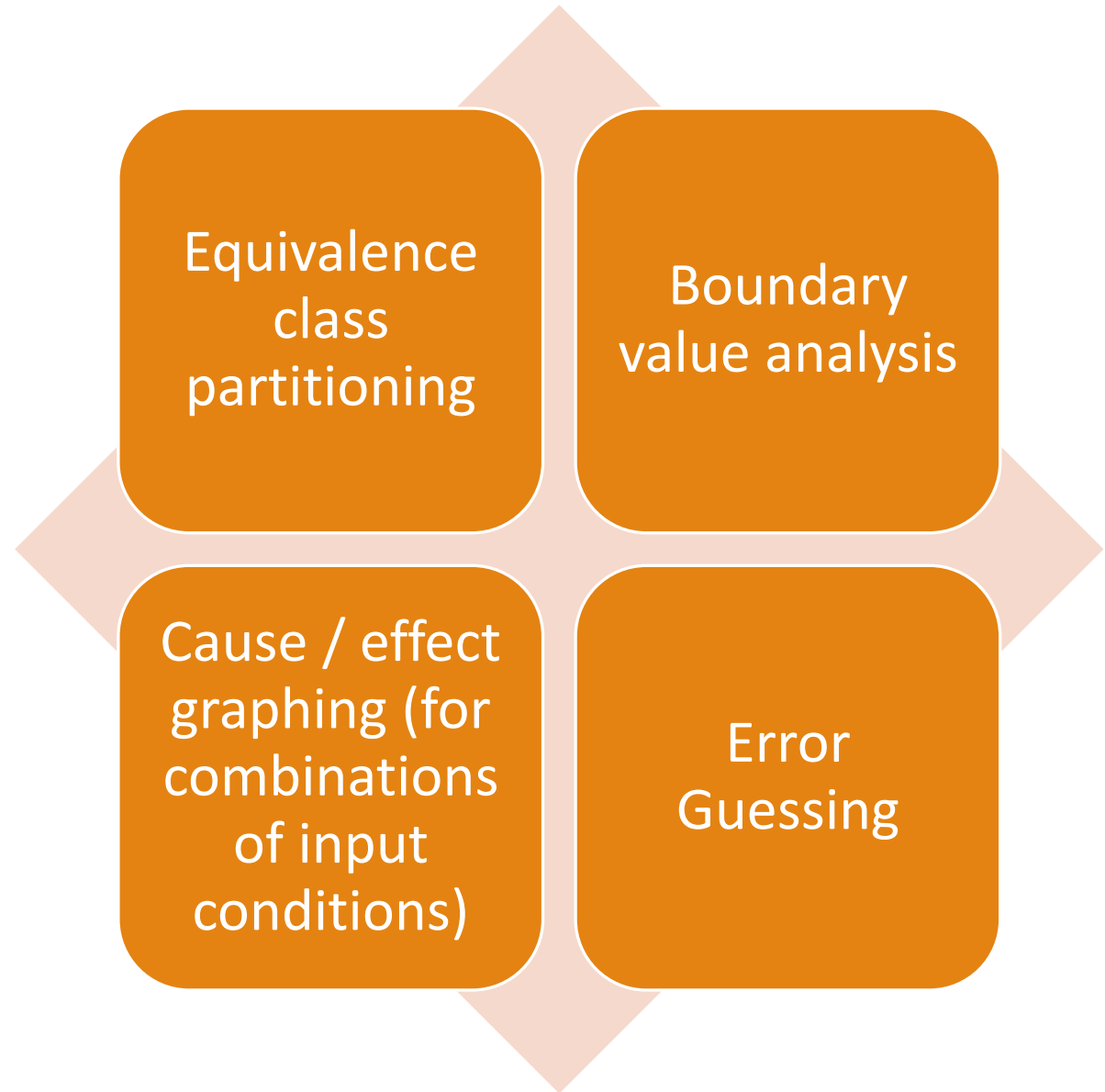
($\langle x, y, z \rangle$; $z \geq x+y$)

Black Box Testing

- Types of errors regarding functional requirements of software:
 - -- Incorrect or missing functions
 - -- Interface errors
 - -- Error in data structure & external data base access
 - -- Performance errors
 - -- Initialization & termination errors
- No functional requirements NO Black Box Testing.
- Demonstrates that each function is fully operational.
- Uncovers different kind of errors than white box testing.
- Performed later in the testing process.

- ◆ Black box techniques derive a set of test cases that satisfy the following criteria:
 - (1) Test cases reduce the number of additional test cases that must be designed to achieve reasonable testing
 - (2) Test cases that tell us something about the presence or absence of classes of errors, rather than errors associated only with the specific test at hand.
- ◆ Black box techniques can supplement the test cases generated by white box.

How to Design Test Cases?



Equivalence Partitioning

- Type of Black Box Testing.
- select the right subset-the subset with the highest probability of finding the most errors.
- The input domain of a program is partitioned into a finite number of equivalence classes such that we can reasonably assume that a test of a representative value of each class is equivalent to a test of any other value.
- By identifying this as an equivalence class, we are stating that if no error is found by a test of one element of the set, it is unlikely that an error would be found by a test of another element of the set.

Traveller Details

File Help

Name: John Smith

Age: 52

Destination: Liverpool ▼

OK Cancel

Fare Price: £42.30

Example

Travel service offers discounts to travelers based on their age.

- 0-4 years 100%
- 5-15 years 50%
- 16-64 years 0%
- 64 years and older 25%

Equivalence classes for age

- 0, 1, 2, 3, 4
- 5, 6, 7, ...15
- 16, 17, 18, ...64
- 65, 66, 67, ...120

Similarly for destinations

- e.g., destination can be grouped based on regions (that have same fair)

Nothing special for name

Identifying Equivalence classes

- ❖ A key concept in the **identification of classes** is **negation**, i.e. If a **characteristic** is **identified** as an **equivalence class**, then one should immediately **negate the characteristic** in order to find **examples of classes** which should **cause the module** to do **something different** such as “generate an error message”.
- ❖ **Partitioning each input condition** into **two or more** groups.
- ❖ **Two types** of equivalence classes are identified:
 1. **Valid** Equivalence Classes (VEC)
 2. **Invalid** Equivalence Classes (IEC)

Why Use Equivalence Partitioning?

- Reduces number of test cases
- Increases test efficiency
- Ensures broad input coverage

Identifying the Equivalence Classes

We have valid Ecs (VEC) and invalid ECs (IEC)

- Design tests such that each valid EC and each invalid EC are included in at least one test case
- Test cases are based on the specifications not on code
- Better results are obtained in testing boundary values of ECs as well.
- Very important to test boundaries. Rich source of very common errors.
- More later on this.

Identifying the Equivalence Classes

- If an input condition specifies a range of values (for example, “the item count can be from 1 to 999”), identify one valid equivalence class ($1 < \text{item count} < 999$) and two invalid equivalence classes ($\text{item count} < 1$ and $\text{item count} > 999$).
- If an input condition specifies the number of values (for example, “one through six owners can be listed for the automobile”), identify one valid equivalence class and two invalid equivalence classes (no owners and more than six owners).

Identifying the Equivalence Classes

If an input condition specifies a set of input values and there is reason to believe that the program handles each differently (e.g. “type of vehicle must be BUS, TRUCK, TAXICAB, PASSENGER, or MOTORCYCLE”), identify a valid equivalence class for each and one invalid equivalence class (“TRAILER,” for example).

- If an input condition specifies a “must be” situation, such as “first character of the identifier must be a letter,” identify one valid equivalence class (it is a letter) and one invalid equivalence class (it is not a letter)

Test Case and the Equivalence Classes

- Formally, one test case per equivalence class should be enough.
- A black box method aimed at increasing the efficiency of testing and, at the same time, improving coverage of potential error conditions

Test Case and the Equivalence Classes

According to the equivalence class partitioning method:

- Each valid EC (VEC) and each invalid EC (IEC) are included in at least one test case.
- Note: equivalence CLASS!
- For an acceptable range of input integers is 1-10, we need at least one test with a value in the range 1-10.
- Boundaries: 1 and 10
- Invalid EC classes might be EC with values < 1 (too low), another class > 10 (too high), another one - negative numbers,

Definition of test cases is done separately for the valid (VEC) and invalid ECs (IEC).

- Note: Run tests for invalid equivalence classes one at a time.

Weak/Strong Equivalence Classes

For an example SUT (system under test), suppose there are **three** input variables from three domains:
A, B, C

$A = A_1 \cup A_2 \cup A_3 \cup \dots \cup A_m$ where $a_i \in A$

$B = B_1 \cup B_2 \cup B_3 \cup \dots \cup B_n$ where $b_j \in B$

$C = C_1 \cup C_2 \cup C_3 \cup \dots \cup C_o$ where $c_k \in C$

Weak Equivalence Class Testing: (Goal: Cover all equivalence classes at least once.)

- **Strategy: Choosing one variable value from each equivalence class** (one a_i , one b_j , and one c_k) such that all classes are covered.
- **# of test cases? = $\max(|A|, |B|, |C|)$** (i.e., the maximum number of equivalence classes among A, B, and C)

Strong Equivalence Class Testing:

- **Strategy: All combinations of class partitions=>based on the Cartesian product of the partition subsets** ($A \times B \times C$),
- i.e., testing all interactions of all equivalence classes. Every possible tuple (a_i, b_j, c_k)
- **# of test cases? = $|A| \times |B| \times |C|$**

Example of Weak Equivalence Testing Class (WETC) *

Number of WETCs need= Max number of equivalence classes among {A, B, C}

Given: $|A| = 3$, $|B| = 4$, $|C| = 2$

4 WETCs are enough

Test Case	A	B	C
WETC1	a1	b1	c1
WETC2	a2	b2	c2
WETC3	a3	b3	c1
WETC4	a1	b4	c2

Example of Strong Equivalence Class Testing (SECT)

$|A| = 3,$

$|B| = 4,$

$|C| = 2$

of test cases

- $3 \times 4 \times 2 = 24$

Test Case	A	B	C
SETC1	a1	b1	c1
SETC2	a1	b1	c2
SETC3	a1	b2	c1
SETC4	a1	b2	c2
SETC5	a1	b3	c1
SETC6	a1	b3	c2
SETC7	a1	b4	c1
SETC8	a1	b4	c2
SETC9	a2	b1	c1
SETC10	a2	b1	c2
SETC11	a2	b2	c1
SETC12	a2	b2	c2
SETC13	a2	b3	c1
SETC14	a2	b3	c2
SETC15	a2	b4	c1
SETC16	a2	b4	c2
SETC17	a3	b1	c1
SETC18	a3	b1	c2
SETC19	a3	b2	c1
SETC20	a3	b2	c2
SETC21	a3	b3	c1
SETC22	a3	b3	c2
SETC23	a3	b4	c1
SETC24	a3	b4	c2

Testing for Robustness

If error conditions are a high priority, we should extend strong equivalence class testing to include **both valid** (E) and **invalid** inputs (U)

- For example: $\text{age} < 0$ and $\text{age} > 120$

Again, robustness can be applied with WECT and SECT

NextDate Example

NextDate is a function with three variables: **month, day, year**. It returns the date of the day after the input date.

- Limitation: 1812-2020

Treatment Summary:

- if it is **not the last day of the month**, the next date function will simply increment the day value.
- At **the end of a month**, the next day is 1 and the month is incremented.
- At the **end of the year**, both the day and the month are reset to 1, and the year incremented.
- Finally, **the problem of leap year** makes determining the last day of a month interesting.

Equivalence Classes

Valid Equivalence Classes (VECs)

- $M1 = \{1 \leq \text{month} \leq 12\}$
- $D1 = \{1 \leq \text{day} \leq 31\}$
- $Y1 = \{1812 \leq \text{year} \leq 2020\}$

Poor Equivalence Classes

Valid Equivalence Classes (VECs)

- $M1 = \{1 \leq \text{month} \leq 12\}$
- $D1 = \{1 \leq \text{day} \leq 31\}$
- $Y1 = \{1812 \leq \text{year} \leq 2020\}$

- ▶ Should take into account special cases
 - ▶ Leap year, February

Better classes

Valid Equivalence Classes

- $M1 = \{1, 3, 5, 7, 8, 10, 12\}$
- $M2 = \{4, 6, 9, 11\}$
- $M3 = \{2\}$
- $D1 = \{1 \leq \text{day} \leq 28\}$
- $D2 = \{29\}$
- $D3 = \{30\}$
- $D4 = \{31\}$
- $Y1 = \{\text{year} = 1900\}$
- $Y2 = \{1812 \leq \text{year} \leq 2020 \text{ AND } (\text{year} \neq 1900) \text{ AND } (\text{year} \bmod 4 = 0)\}$
- $Y3 = \{1812 \leq \text{year} \leq 2020 \text{ AND } \text{year} \bmod 4 \neq 0\}$

Weak Equivalent Class - Test cases

#test cases=maximum partition size (D)=4

Test Case ID	Month	Day	Year	Output
WETC1	6	14	1900	6/15/1900
WETC2	7	29	1912	7/30/1912
WETC3	2	30	1913	Invalid Input date (not possible)
WETC4	6	31	1900	Invalid Input date

Strong Equivalent Class Testing - Test

cases

- #SECT test cases= partition size (D) x partition size (M) x partition size (Y) = 3x4x3=36 test cases

Test Case ID	Month	Day	Year	Expected Output
SE1	6	14	1900	6/15/1900
SE2	6	14	1912	6/15/1912
SE3	6	14	1913	6/15/1913
SE4	6	29	1900	6/30/1900
SE5	6	29	1912	6/30/1912
SE6	6	29	1913	6/30/1913
SE7	6	30	1900	7/1/1900
SE8	6	30	1912	7/1/1912
SE9	6	30	1913	7/1/1913
SE10	6	31	1900	ERROR
SE11	6	31	1912	ERROR
SE12	6	31	1913	ERROR
SE13	7	14	1900	7/15/1900
SE14	7	14	1912	7/15/1912
SE15	7	14	1913	7/15/1913
SE16	7	29	1900	7/30/1900
SE17	7	29	1912	7/30/1912

SE18	7	29	1913	7/30/1913
SE19	7	30	1900	7/31/1900
SE20	7	30	1912	7/31/1912
SE21	7	30	1913	7/31/1913
SE22	7	31	1900	8/1/1900
SE23	7	31	1912	8/1/1912
SE24	7	31	1913	8/1/1913
SE25	2	14	1900	2/15/1900
SE26	2	14	1912	2/15/1912
SE27	2	14	1913	2/15/1913
SE28	2	29	1900	ERROR
SE29	2	29	1912	3/1/1912
SE30	2	29	1913	ERROR
SE31	2	30	1900	ERROR
SE32	2	30	1912	ERROR
SE33	2	30	1913	ERROR
SE34	2	31	1900	ERROR
SE35	2	31	1912	ERROR
SE36	2	31	1913	ERROR ₁

